

LAPORAN TUGAS 2
IMPLEMENTASI ALGORITMA GENETIKA UNTUK PENCARIAN KATA
DALAM KAMUS BAHASA DAERAH

Bahasa Daerah: Bahasa Makassar



Zulhar Walikram

105841109721

Teknik Informatika

1. Tujuan

Program ini mengimplementasikan Algoritma Genetika (Genetic Algorithm/GA) untuk mencari kata yang paling mendekati kata target pada database kamus Bahasa Makassar. Proses minimal satu generasi meliputi populasi, fitness, seleksi roulette wheel, crossover, mutasi, dan evaluasi populasi baru.

2. Dataset Kamus

No	Kata	Arti
1	tabe	permisi / salam sopan
2	iye	iya
3	tena	tidak
4	agang	teman
5	balla	rumah
6	juku	ikan
7	je'ne	air
8	manre	makan
9	tau	orang / manusia
10	baji	baik / bagus
11	ammaca	membaca
12	appa	apa

3. Representasi Kromosom

Satu kromosom direpresentasikan sebagai indeks integer yang menunjuk ke satu kata pada database. Representasi ini membuat populasi mudah diproses oleh operasi seleksi, crossover, dan mutasi.

4. Perhitungan Fitness

Fitness menggabungkan jarak karakter, perbedaan panjang kata, dan proporsi karakter yang sama pada posisi yang sama. Rumus program:

$$\text{Fitness} = 1 / (1 + \text{jarak} + \text{penalti_panjang}) + 0,5 \times \text{kemiripan_posisi}$$

5. Contoh Perhitungan Fitness

Kandidat	Jarak	Posisi sama	Kemiripan	Fitness
manre	0	5	1.00	1.500000
tabe	4	1	0.20	0.266667
tena	4	1	0.20	0.266667
baji	4	1	0.20	0.266667
ammaca	6	0	0.00	0.125000

6. Seleksi Roulette

Setiap individu mendapatkan peluang seleksi berdasarkan fitness relatif terhadap total fitness populasi. Rumusnya:

$$\text{Probabilitas seleksi}(i) = \text{Fitness}(i) / \text{Total Fitness}$$

7. Crossover

Karena kromosom proyek ini terdiri dari satu gen berupa indeks kata, crossover diterapkan dengan memilih gen dari salah satu parent. Probabilitas crossover default adalah 0,80.

8. Mutasi

Mutasi dilakukan dengan probabilitas default 0,20. Jika mutasi terjadi, indeks kata diganti dengan indeks kata lain secara acak dari database.

9. Hasil Generasi ke-1

Pengujian menggunakan random seed 42 agar hasil dapat direproduksi. Parameter: target = 'manre', populasi = 6, crossover = 0,80, mutasi = 0,20, dan generasi = 1.

```

001 import random
002
003 # =====
004 # TUGAS 2 - ALGORITMA GENETIKA
005 # Kamus Bahasa Daerah (Bahasa Makassar)
006 # =====
007
008 KAMUS = [
009     {"kata": "tabe", "arti": "permisi / salam sopan"},
010     {"kata": "iye", "arti": "iya"},
011     {"kata": "tena", "arti": "tidak"},
012     {"kata": "agang", "arti": "teman"},
013     {"kata": "balla", "arti": "rumah"},
014     {"kata": "juku", "arti": "ikan"},
015     {"kata": "je'ne", "arti": "air"},
016     {"kata": "manre", "arti": "makan"},
017     {"kata": "tau", "arti": "orang / manusia"},
018     {"kata": "baji", "arti": "baik / bagus"},
019     {"kata": "ammaca", "arti": "membaca"},
020     {"kata": "appa", "arti": "apa"},
021 ]
022
023 # -----
024 # Utilitas
025 # -----
026 def normalisasi(teks):
027     return teks.lower().strip()
028
029 def hamming_padded(a, b):
030     """Jarak sederhana berbasis karakter dengan padding."""
031     n = max(len(a), len(b))
032     a = a.ljust(n)
033     b = b.ljust(n)
034     return sum(x != y for x, y in zip(a, b))
035
036 def fitness_kata(kandidat, target):
037     """
038     Fitness = 1 / (1 + jarak)
039     Ditambah bonus proporsi karakter yang sama pada posisi yang sama.
040     Nilai lebih besar = kandidat lebih dekat dengan target.
041     """
042     kandidat = normalisasi(kandidat)
043     target = normalisasi(target)
044
045     if not target:
046         return 0.0
047
048     jarak = hamming_padded(kandidat, target)
049     posisi_sama = sum(
050         1 for i in range(min(len(kandidat), len(target)))
051         if kandidat[i] == target[i]
052     )
053     kemiripan_posisi = posisi_sama / len(target)
054
055     # Penalti panjang agar kata dengan panjang sangat berbeda tidak unggul.
056     penalti_panjang = abs(len(kandidat) - len(target))
057     return (1 / (1 + jarak + penalti_panjang)) + (0.5 * kemiripan_posisi)
058
059 def buat_individu():
060     return random.randrange(len(KAMUS))
061
062 def tampilkan_kamus():
063     print("\n=== KAMUS BAHASA MAKASSAR ===")
064     print("-" * 48)
065     print(f'{"No":<4}{"Kata":<15}{"Arti":<15}')
066     print("-" * 48)
067     for i, item in enumerate(KAMUS, 1):
068         print(f'{i:<4}{item["kata"]:<15}{item["arti"]:<15}')
069     print("-" * 48)
070

```

Gambar 1. Screenshot bagian awal source code program.

10. Output Pengujian Program

=== PROSES ALGORITMA GENETIKA ===

Target: manre

Ukuran populasi: 6

Prob. Crossover: 0.8

Prob. mutase: 0.2

Jumlah generasi: 1

=== POPULASI AWAL ===

Ind	Kromoson	Kata	Fitness
-----	----------	------	---------

1	10	ammaca	0.125000
2	1	iye	0.125000
3	0	tabe	0.266667
4	11	appa	0.142857
5	4	balla	0.300000
6	3	agang	0.166667

--- GENERASI 1 ---

Total fitness = 1.126190

Hasil Seleksi Roulette:

Parent 1: tabe (kromosom 0)

Parent 2: balla (kromosom 4)

Parent 3: balla (kromosom 4)

Parent 4: agang (kromosom 3)

Parent 5: ammaca (kromosom 10)

Parent 6: tabe (kromosom 0)

CROSSOVER:

Pasangan 1-2: tabe x balla -> tabe, balla

Pasangan 3-4: balla x agang -> balla, agang

Pasangan 5-6: ammaca x tabe -> tabe, ammaca

MUTASI:

Anak 1: tabe (tidak berubah)

Anak 2: balla (tidak berubah)

Anak 3: balla (tidak berubah)

Anak 4: agang (tidak berubah)

Anak 5: tabe -> agang

Anak 6: ammaca (tidak berubah)

=== POPULASI BARU GENERASI 1 ===

Ind	Kromoson	Kata	Fitness
-----	----------	------	---------

1	0	tabe	0.266667
2	4	balla	0.300000
3	4	balla	0.300000
4	3	agang	0.166667
5	3	agang	0.166667
6	10	ammaca	0.125000

Terbaik generasi 1: balla = rumah (fitness 0.300000)

11. Struktur Menu

- 1. Tampilkan Kamus
- 2. Cari Kata
- 3. Jalankan Algoritma Genetika
- 4. Tampilkan Populasi
- 5. Hasil Fitness
- 6. Seleksi Roulette
- 7. Cross Over
- 8. Mutasi
- 9. Generasi Baru
- 10. Keluar

12. Kesimpulan

Program telah memenuhi alur utama tugas: database kamus minimal 10 kata, perhitungan fitness, seleksi roulette, crossover, mutasi, dan evaluasi populasi baru minimal satu generasi. Dataset dan parameter GA dapat dikembangkan lebih lanjut untuk eksperimen dengan kata serta generasi yang lebih banyak.

13. Cara Menjalankan

Buka terminal pada folder source code, kemudian jalankan:

```
python kamus_bahasa_makassar_ga.py
```

Pilih menu 1 untuk menampilkan kamus, menu 2 untuk pencarian langsung, atau menu 3/9 untuk menjalankan proses Algoritma Genetika.

14. Source Code Lengkap

```
import random
```

```
# =====  
# TUGAS 2 - ALGORITMA GENETIKA  
# Kamus Bahasa Daerah (Bahasa Makassar)  
# =====
```

```
KAMUS = [  
    {"kata": "tabe", "arti": "permisi / salam sopan"},  
    {"kata": "iye", "arti": "iya"},  
    {"kata": "tena", "arti": "tidak"},  
    {"kata": "agang", "arti": "teman"},  
    {"kata": "balla", "arti": "rumah"},
```

```

{"kata": "juku", "arti": "ikan"},
{"kata": "je'ne", "arti": "air"},
{"kata": "manre", "arti": "makan"},
{"kata": "tau", "arti": "orang / manusia"},
{"kata": "baji", "arti": "baik / bagus"},
{"kata": "ammaca", "arti": "membaca"},
{"kata": "appa", "arti": "apa"},
]

```

```

# -----
# Utilitas
# -----

```

```

def normalisasi(teks):
    return teks.lower().strip()

```

```

def hamming_padded(a, b):
    """Jarak sederhana berbasis karakter dengan padding."""
    n = max(len(a), len(b))
    a = a.ljust(n)
    b = b.ljust(n)
    return sum(x != y for x, y in zip(a, b))

```

```

def fitness_kata(kandidat, target):
    """
    Fitness = 1 / (1 + jarak)
    Ditambah bonus proporsi karakter yang sama pada posisi yang sama.
    Nilai lebih besar = kandidat lebih dekat dengan target.
    """
    kandidat = normalisasi(kandidat)
    target = normalisasi(target)

```

```

    if not target:
        return 0.0

```

```

    jarak = hamming_padded(kandidat, target)
    posisi_sama = sum(
        1 for i in range(min(len(kandidat), len(target)))
        if kandidat[i] == target[i]
    )
    kemiripan_posisi = posisi_sama / len(target)

```

```

    # Penalti panjang agar kata dengan panjang sangat berbeda tidak unggul.
    penalti_panjang = abs(len(kandidat) - len(target))
    return (1 / (1 + jarak + penalti_panjang)) + (0.5 * kemiripan_posisi)

```

```

def buat_individu():
    return random.randrange(len(KAMUS))

def tampilkan_kamus():
    print("\n=== KAMUS BAHASA MAKASSAR ===")
    print("-" * 48)
    print(f'{"No":<4}{"Kata":<15}{"Arti"}')
    print("-" * 48)
    for i, item in enumerate(KAMUS, 1):
        print(f'{"i":<4}{"item['kata']":<15}{"item['arti']}")
    print("-" * 48)

def cari_kata():
    target = input("Masukkan kata yang ingin dicari: ")
    target = normalisasi(target)

    hasil = [x for x in KAMUS if normalisasi(x["kata"]) == target]

    if hasil:
        print(f"\nDitemukan: {hasil[0]['kata']} = {hasil[0]['arti']}")
    else:
        print("\nKata tidak ditemukan pada database.")
        print("Gunakan menu 3 untuk mencari kata yang paling mirip dengan GA.")

def buat_populasi(ukuran):
    return [buat_individu() for _ in range(ukuran)]

def evaluasi(populasi, target):
    return [(ind, fitness_kata(KAMUS[ind]["kata"], target)) for ind in populasi]

def tampilkan_populasi(populasi, target, judul="POPULASI"):
    print(f"\n=== {judul} ===")
    print(f'{"Ind":<5}{"Kromosom":<10}{"Kata":<12}{"Fitness"}')
    print("-" * 42)
    data = evaluasi(populasi, target)
    for i, (ind, fit) in enumerate(data, 1):
        print(f'{"i":<5}{"ind":<10}{"KAMUS[ind]['kata']":<12}{"fit:.6f}")
    return data

def seleksi_roulette(populasi, target):
    data = evaluasi(populasi, target)
    total = sum(fit for _, fit in data)

```

```
if total == 0:  
    return random.choice(populasi)
```

```
r = random.uniform(0, total)  
kumulatif = 0.0
```

```
for ind, fit in data:  
    kumulatif += fit  
    if r <= kumulatif:  
        return ind
```

```
return data[-1][0]
```

```
def crossover(parent1, parent2):
```

```
    """
```

```
    Karena kromosom berupa indeks kata (satu gen), crossover menggunakan  
    one-point-like choice: anak mengambil gen dari salah satu parent.
```

```
    """
```

```
    if random.random() < 0.5:  
        return parent1, parent2  
    return parent2, parent1
```

```
def mutasi(individu, probabilitas=0.20):
```

```
    if random.random() < probabilitas:  
        pilihan = list(range(len(KAMUS)))  
        pilihan.remove(individu)  
        individu = random.choice(pilihan)  
        return individu, True  
    return individu, False
```

```
def jalankan_ga(target, ukuran_populasi=6, pc=0.80, pm=0.20, generasi=1):
```

```
    print("\n=== PROSES ALGORITMA GENETIKA ===")  
    print(f"Target          : {target}")  
    print(f"Ukuran populasi    : {ukuran_populasi}")  
    print(f"Prob. crossover     : {pc}")  
    print(f"Prob. mutasi        : {pm}")  
    print(f"Jumlah generasi     : {generasi}")
```

```
    populasi = buat_populasi(ukuran_populasi)  
    tampilkan_populasi(populasi, target, "POPULASI AWAL")
```

```
    for gen in range(1, generasi + 1):  
        data = evaluasi(populasi, target)  
        total_fitness = sum(f for _, f in data)
```

```

print(f"\n--- GENERASI {gen} ---")
print(f"Total fitness = {total_fitness:.6f}")

parents = []
for _ in range(ukuran_populasi):
    p = seleksi_roulette(populasi, target)
    parents.append(p)

print("\nHasil Seleksi Roulette:")
for i, p in enumerate(parents, 1):
    print(f"Parent {i}: {KAMUS[p]['kata']} (kromosom {p})")

offspring = []
print("\nCROSSOVER:")
for i in range(0, ukuran_populasi - 1, 2):
    p1, p2 = parents[i], parents[i + 1]
    if random.random() < pc:
        c1, c2 = crossover(p1, p2)
        print(
            f"Pasangan {i+1}-{i+2}: "
            f"{KAMUS[p1]['kata']} x {KAMUS[p2]['kata']} "
            f"-> {KAMUS[c1]['kata']}, {KAMUS[c2]['kata']}"
        )
    else:
        c1, c2 = p1, p2
        print(
            f"Pasangan {i+1}-{i+2}: tidak crossover "
            f"-> {KAMUS[c1]['kata']}, {KAMUS[c2]['kata']}"
        )
    offspring.extend([c1, c2])

if len(parents) % 2 == 1:
    offspring.append(parents[-1])

print("\nMUTASI:")
hasil_mutasi = []
for i, child in enumerate(offspring, 1):
    before = child
    child, berubah = mutasi(child, pm)
    hasil_mutasi.append(child)
    if berubah:
        print(f"Anak {i}: {KAMUS[before]['kata']} -> {KAMUS[child]['kata']}")
    else:

```

```

        print(f'Anak {i}: {KAMUS[child]['kata']} (tidak berubah)')

    populasi = hasil_mutasi
    data_baru = tampilkan_populasi(
        populasi, target, f"POPULASI BARU GENERASI {gen}"
    )

    terbaik = max(data_baru, key=lambda x: x[1])
    best_idx, best_fit = terbaik
    print(
        f"\nTerbaik generasi {gen}: "
        f"{KAMUS[best_idx]['kata']} = {KAMUS[best_idx]['arti']} "
        f"(fitness {best_fit:.6f})"
    )

    return populasi

def tampilkan_fitness():
    target = normalisasi(input("Masukkan kata target untuk evaluasi fitness: "))
    print("\n=== HASIL FITNESS ===")
    for item in KAMUS:
        f = fitness_kata(item["kata"], target)
        print(f'{item["kata"]:<10} fitness = {f:.6f}')

def menu_utama():
    while True:
        print("\n=====")
        print("=== KAMUS BAHASA MAKASSAR + GA ===")
        print("=====")
        print("1. Tampilkan Kamus")
        print("2. Cari Kata")
        print("3. Jalankan Algoritma Genetika")
        print("4. Tampilkan Populasi")
        print("5. Hasil Fitness")
        print("6. Seleksi Roulette")
        print("7. Cross Over")
        print("8. Mutasi")
        print("9. Generasi Baru")
        print("10. Keluar")

        pilihan = input("Pilih menu [1-10]: ").strip()

        if pilihan == "1":
            tampilkan_kamus()

```

```

elif pilihan == "2":
    cari_kata()

elif pilihan == "3":
    target = normalisasi(input("Masukkan kata target: "))
    try:
        ukuran = int(input("Ukuran populasi [default 6]: ") or 6)
        pc = float(input("Probabilitas crossover [default 0.8]: ") or 0.8)
        pm = float(input("Probabilitas mutasi [default 0.2]: ") or 0.2)
        generasi = int(input("Jumlah generasi [minimal 1]: ") or 1)
        if generasi < 1:
            generasi = 1
        jalankan_ga(target, ukuran, pc, pm, generasi)
    except ValueError:
        print("Input angka tidak valid.")

elif pilihan == "4":
    target = normalisasi(input("Masukkan kata target: "))
    ukuran = int(input("Ukuran populasi [default 6]: ") or 6)
    random.seed(42)
    populasi = buat_populasi(ukuran)
    tampilkan_populasi(populasi, target)

elif pilihan == "5":
    tampilkan_fitness()

elif pilihan == "6":
    target = normalisasi(input("Masukkan kata target: "))
    random.seed(42)
    populasi = buat_populasi(6)
    data = evaluasi(populasi, target)
    total = sum(f for _, f in data)
    print("\n=== SELEKSI ROULETTE ===")
    print(f'{'Kata':<12}{'Fitness':<12}{'Probabilitas':<15}')
    for ind, fit in data:
        prob = fit / total if total else 0
        print(f'{'KAMUS[ind]['kata']':<12}{'fit':<12.6f}{'prob':<15.6%}')
    terpilih = seleksi_roulette(populasi, target)
    print(f'\nHasil spin roulette: {KAMUS[terpilih]['kata']}')

elif pilihan == "7":
    p1 = input("Parent 1 (kata): ").strip().lower()
    p2 = input("Parent 2 (kata): ").strip().lower()

```

```

indeks1 = next((i for i,x in enumerate(KAMUS) if x["kata"] == p1), None)
indeks2 = next((i for i,x in enumerate(KAMUS) if x["kata"] == p2), None)
if indeks1 is None or indeks2 is None:
    print("Parent tidak ada di database.")
else:
    random.seed(42)
    c1, c2 = crossover(indeks1, indeks2)
    print(f"Parent : {p1} x {p2}")
    print(f"Anak  : {KAMUS[c1]['kata']} x {KAMUS[c2]['kata']}")

```

```

elif pilihan == "8":
    kata = input("Masukkan kata yang akan dimutasi: ").strip().lower()
    indeks = next((i for i,x in enumerate(KAMUS) if x["kata"] == kata), None)
    if indeks is None:
        print("Kata tidak ada di database.")
    else:
        random.seed(42)
        hasil, berubah = mutasi(indeks, 1.0)
        print(f"{kata} -> {KAMUS[hasil]['kata']} (mutasi={'ya' if berubah else 'tidak'})")

```

```

elif pilihan == "9":
    target = normalisasi(input("Masukkan kata target: "))
    random.seed(42)
    jalankan_ga(target, ukuran_populasi=6, pc=0.8, pm=0.2, generasi=1)

```

```

elif pilihan == "10":
    print("Program selesai. Terima kasih.")
    break

```

```

else:
    print("Pilihan tidak tersedia.")

```

```

if __name__ == "__main__":
    menu_utama()

```